

程序设计实习（实验班-2026春）

大数据并行计算模型：MPC

授课教师：姜少峰

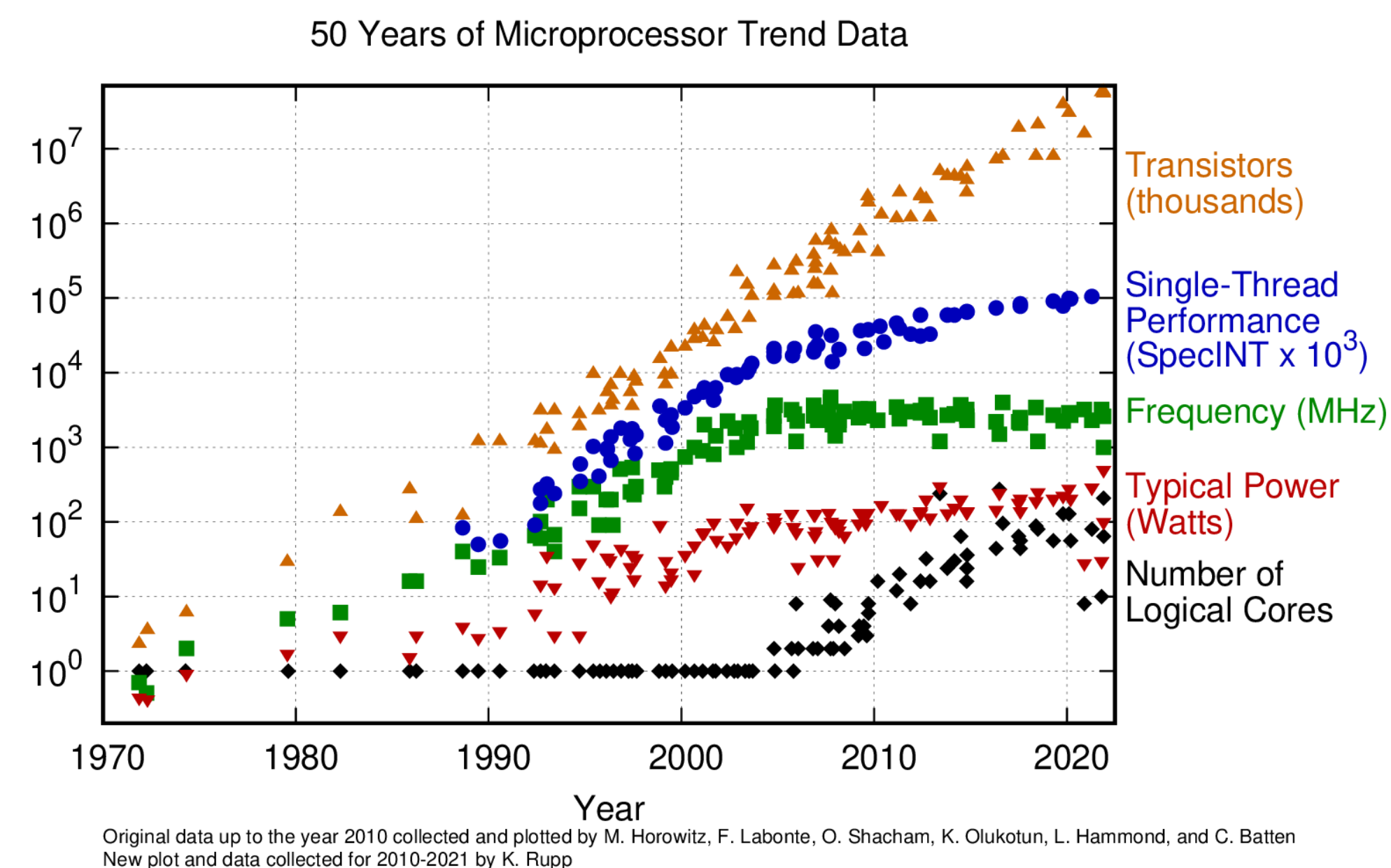
助教：方心童、葛致远、李雷思问、刘鹏飞

Email: shaofeng.jiang@pku.edu.cn

大数据时代

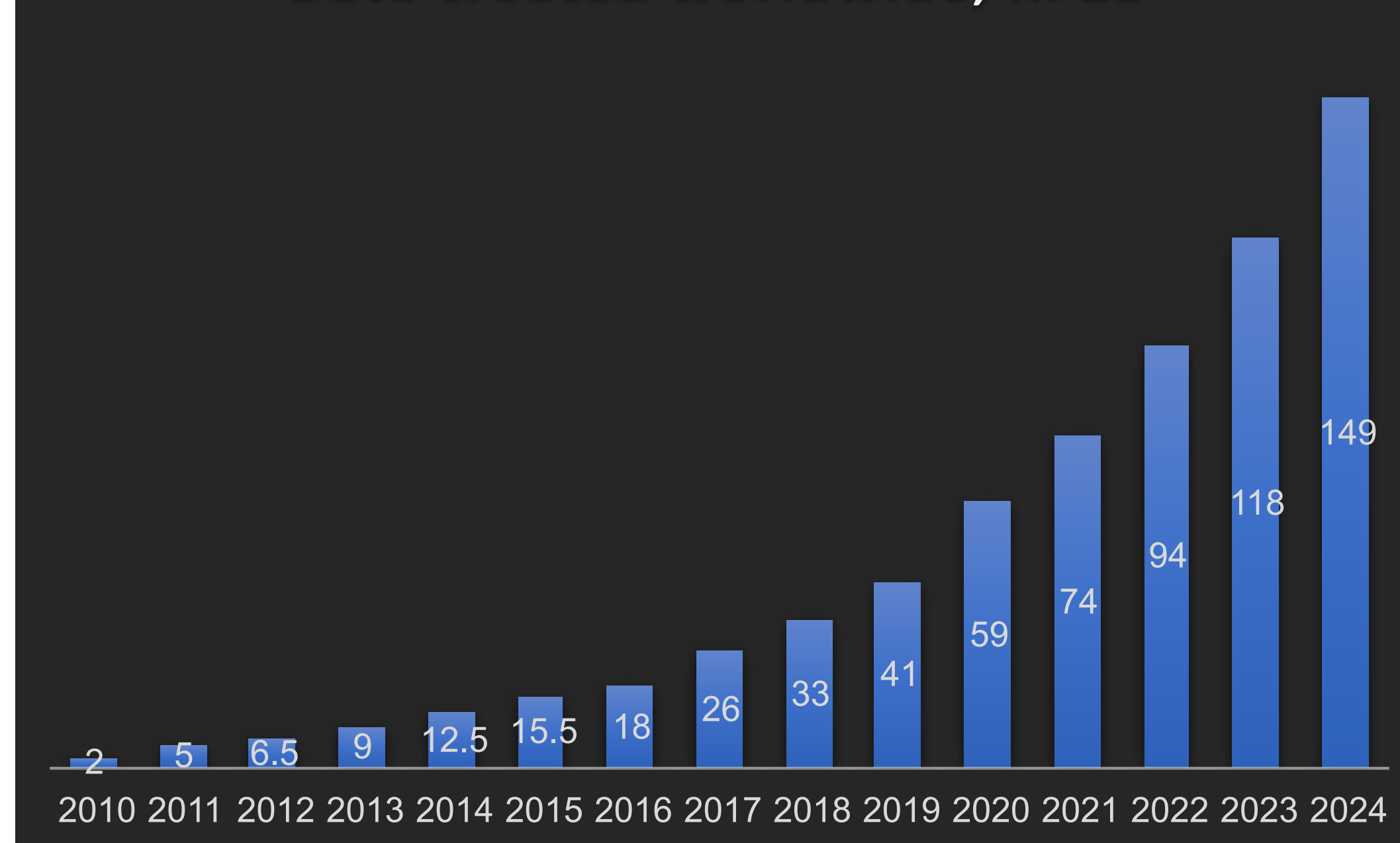
甚至线性时间都不够高效

有效算法 = 多项式时间算法？



单个CPU：每秒 10^{10} 浮点运算；超级计算机：每秒 10^{18} 浮点运算

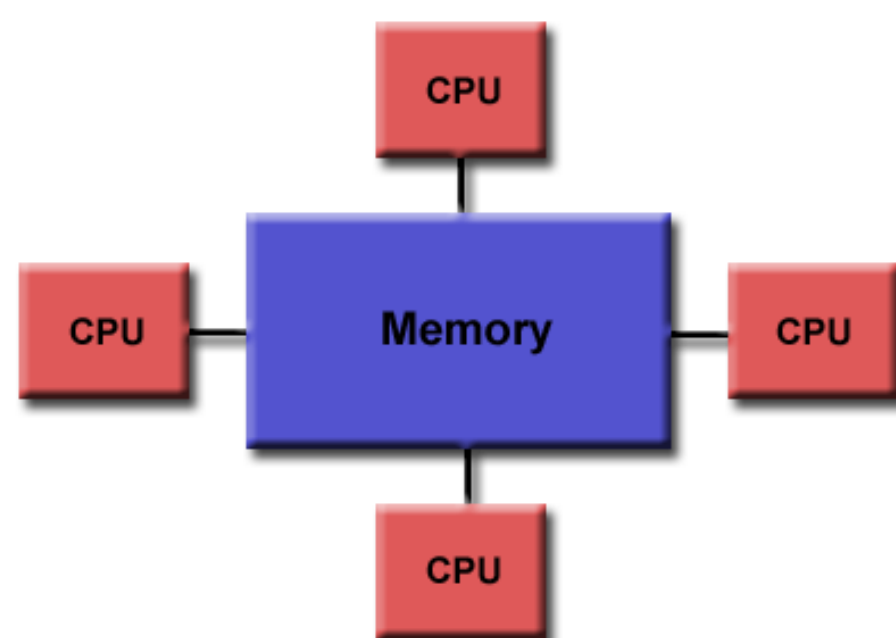
Data created worldwide, in ZB



即使仅扫描一遍数据也不切实际；单CPU性能增速跟不上数据

“亚线性”计算模型

- **数据流算法**：扫一遍数据流，用较小的空间计算
 - 路由器/物联网设备上直接做数据分析
- **并行算法**



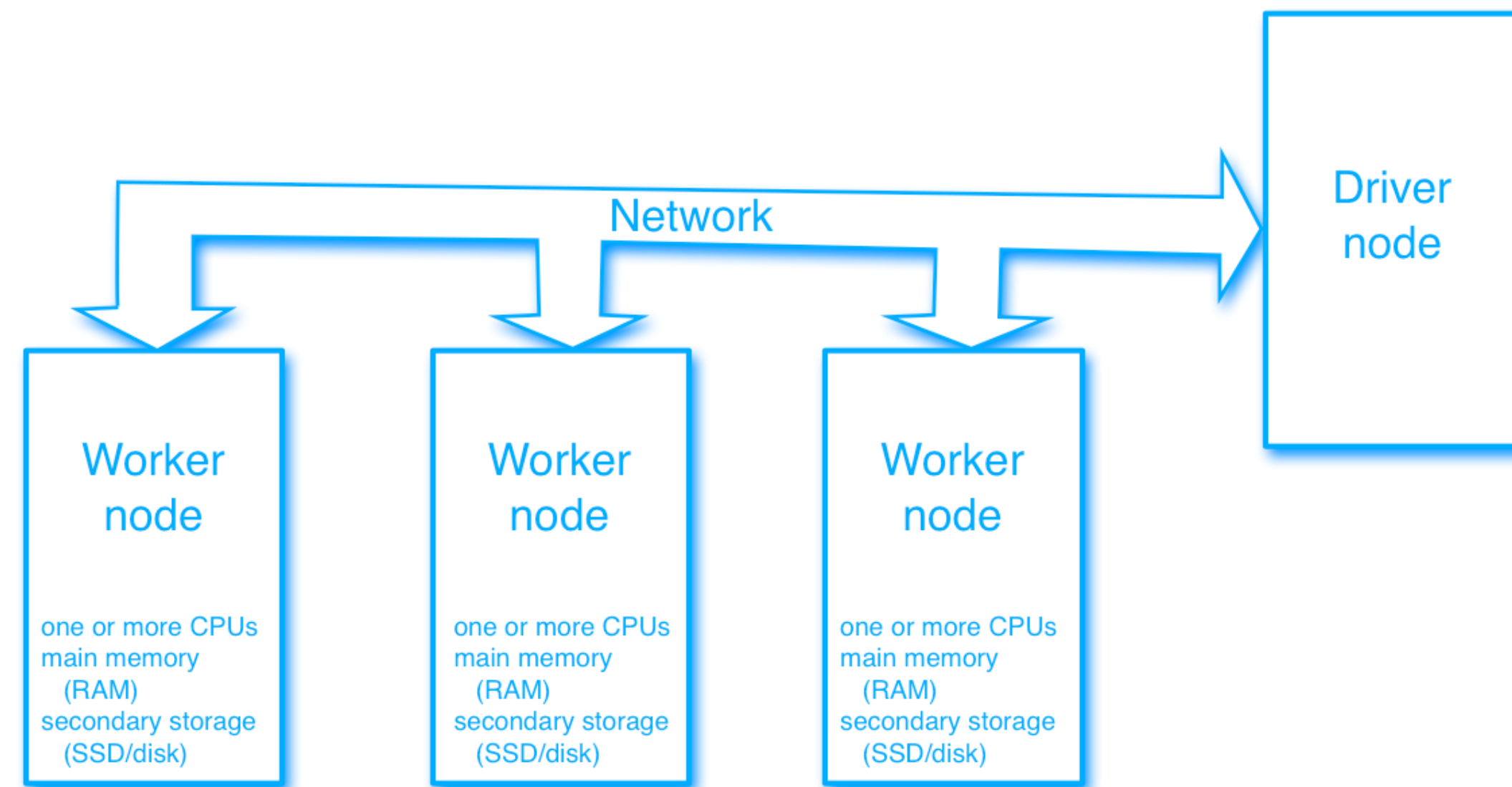
MapReduce框架是分布式计算
最流行的软件实现

MapReduce

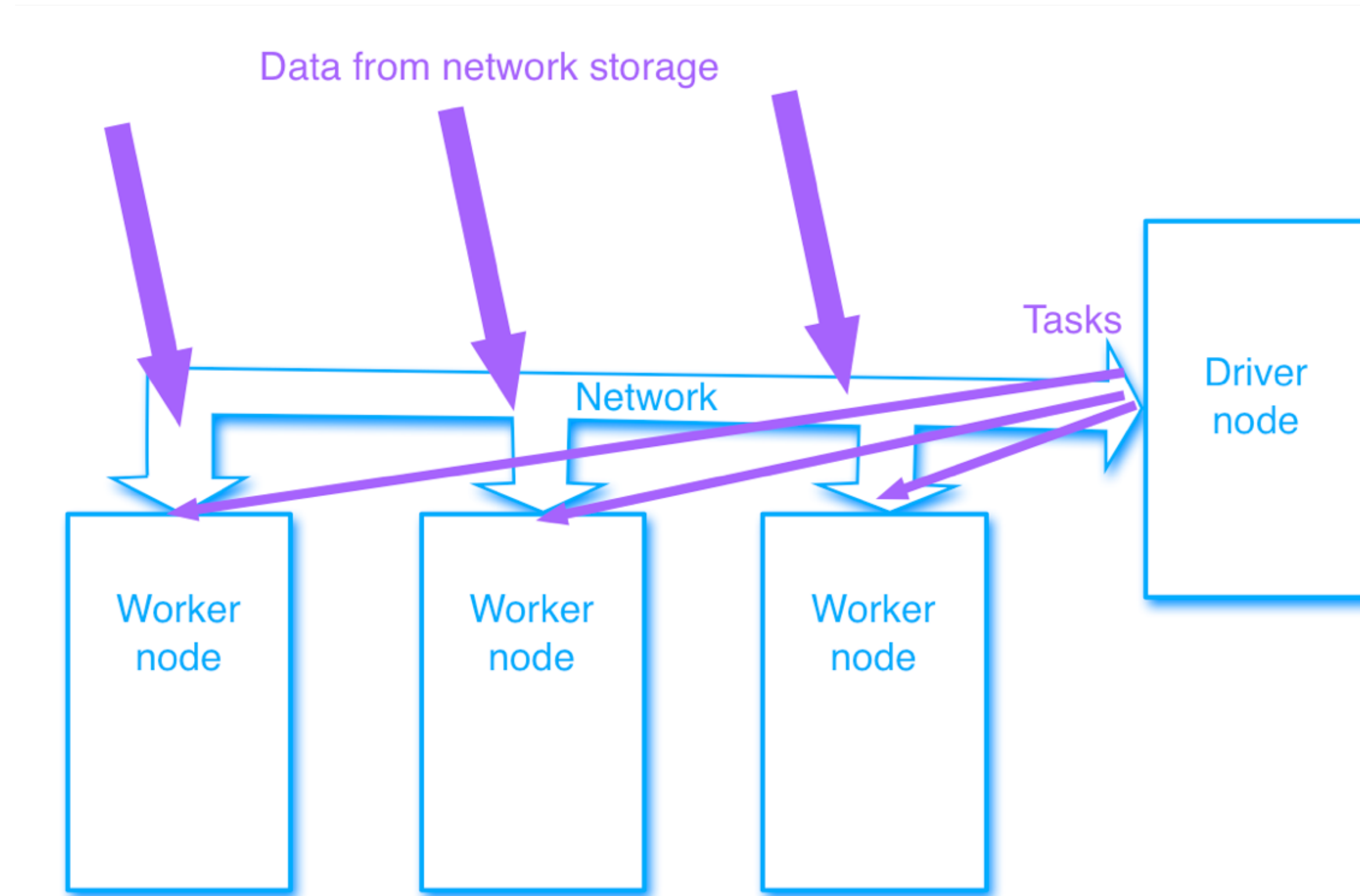
MapReduce：基本设定

- 设有规模 n 的输入数据，这些数据分散存储在若干机器上
- 机器空间和机器个数： $n^{1-\epsilon}$ 亚线性与输入规模 n ，否则trivial/不是大数据
- 在数据集上的计算通过某些抽象的基本操作进行

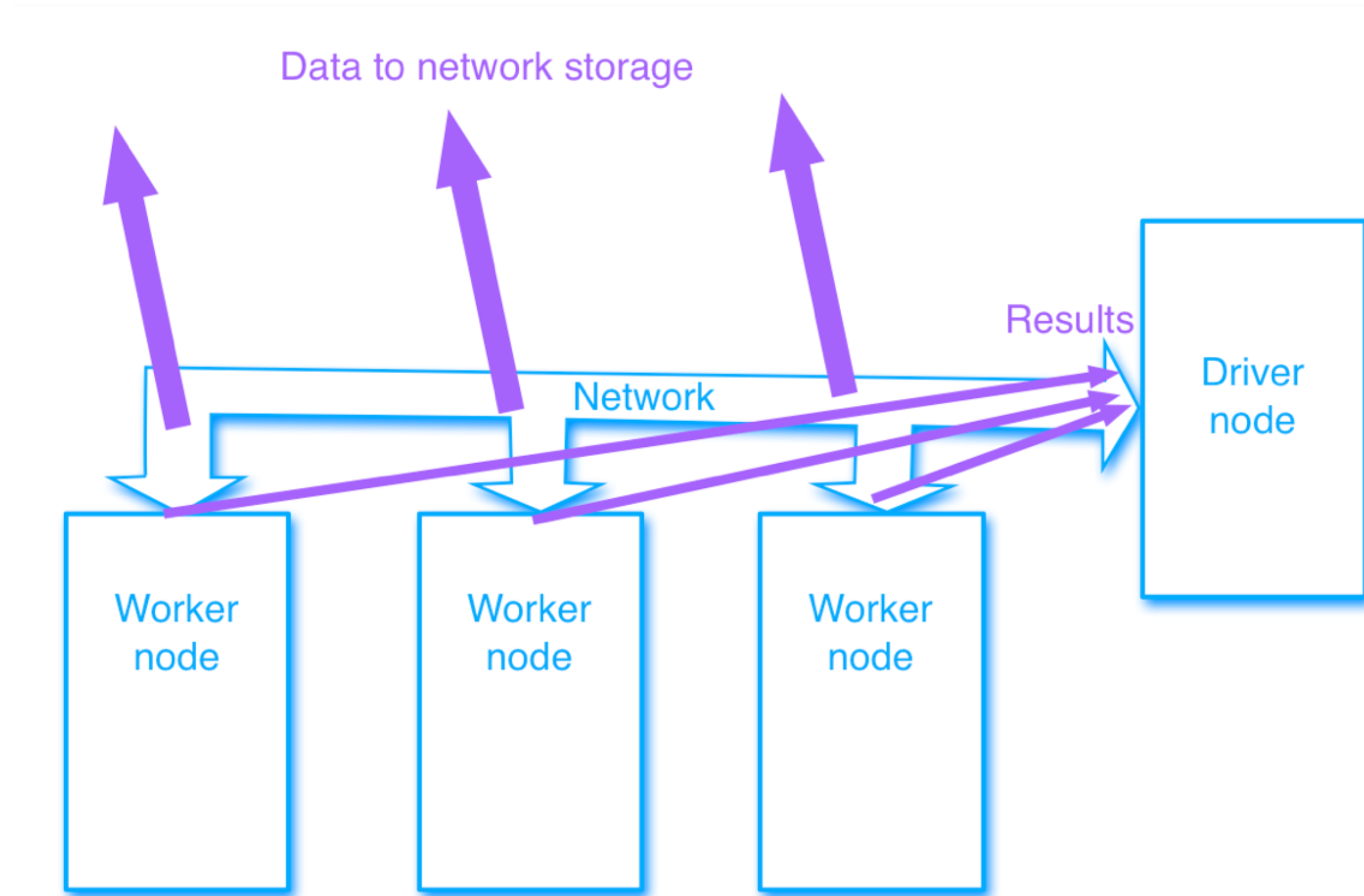
大体架构： Driver + Worker



Driver将数据与任务分配给Worker

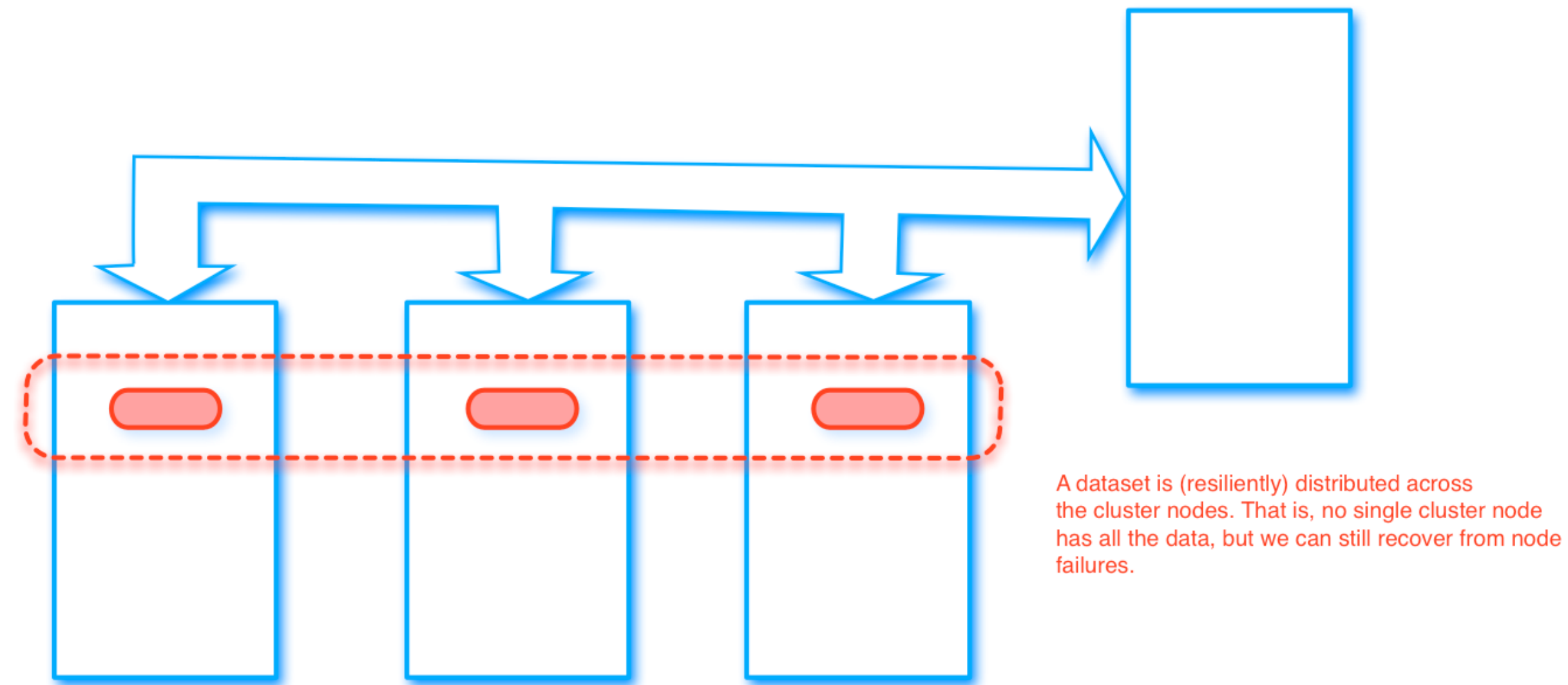


计算完成后Driver从Worker处收集结果



存储： Resilient Distributed Dataset (RDD)

可以理解成分布式的“数组”



MapReduce

- MapReduce框架将RDD抽象出来，定义了若干种RDD上的基本操作
 - 这些基本操作的细节由MapReduce实现，使用者不需要关心
 - 用户不以“分布式数组”的方式操作RDD（虽然RDD的直接抽象是数组）
- 换言之，MapReduce定义了几个基本操作/编程语言，想要操作MapReduce大数据的程序必须要用这几个基本操作写成

一般形式

[Karloff, Suri, Vassilvitskii, SODA 2010]

第一次将MapReduce框架的基本操作描述成数学语言

- 输入：一组 (K, V) 对
- Mapper: 以数据中的每一对 (K, V) 为输入，输出一个或多个 (K, V) 对
- Reducer: 输入所有 $K = K^*$ 的 (K, V) ，返回同样以 K^* 为key的若干 (K, V)
 - 即都是 $(K^*, (V_1, \dots, V_t))$ 形式；但输出通常只包含一个 (K, V)
- 一轮 = Mapper + Reducer在整个数据集进行一次操作

可以理解成把输入的每个 (K, V) 对映射成若干其他 (K, V) 对

例子：统计频数向量的 ℓ_0 -范数

即不同元素的个数

- 输入 n 个int分布式存储在一些机器上，想要计算这些数据的频数向量的 ℓ_0 -范数
- 如何用Mapper和Reducer的语言在 $O(1)$ 轮完成？

- 输入 $\{(K, V) = (i, a_i)\}$

切换关键字

- Mapper: 将每个 (i, a_i) 映射成 $(a_i, 1)$

由于关心 ℓ_0 ，此处将 a_i 所有的出现汇总成1次
若要求 ℓ_p 范数应该输出 (a_i, t^p)

- Reducer: 输入 $K = a_i$ 的所有tuple $(K = a_i, (V_1 = 1, \dots, V_t = 1))$ 输出 $(a_i, 1)$

- Mapper: $(a_i, t) \mapsto (0, t)$, Reducer: $(0, (t_1, \dots, t_m)) \mapsto (0, \sum_i t_i)$

对t求和这一简单操作的MapReduce语言描述

Apache Spark

- Apache Spark是一种非常流行的支持MapReduce的框架

Spark运行模式很灵活，可以尝试在自己单机配置运行，会利用多个CPU核心进行并行计算

Transformations	<code>map</code> <code>(f : T ⇒ U)</code> : <code>RDD[T] ⇒ RDD[U]</code> <code>filter</code> <code>(f : T ⇒ Bool)</code> : <code>RDD[T] ⇒ RDD[T]</code> <code>flatMap</code> <code>(f : T ⇒ Seq[U])</code> : <code>RDD[T] ⇒ RDD[U]</code> <code>sample</code> <code>(fraction : Float)</code> : <code>RDD[T] ⇒ RDD[T]</code> <code>groupByKey</code> <code>()</code> : <code>RDD[(K, V)] ⇒ RDD[(K, Seq[V])]</code> <code>reduceByKey</code> <code>(f : (V, V) ⇒ V)</code> : <code>RDD[(K, V)] ⇒ RDD[(K, V)]</code> <code>union</code> <code>()</code> : <code>(RDD[T], RDD[T]) ⇒ RDD[T]</code> <code>join</code> <code>()</code> : <code>(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (V, W))]</code> <code>cogroup</code> <code>()</code> : <code>(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (Seq[V], Seq[W]))]</code> <code>crossProduct</code> <code>()</code> : <code>(RDD[T], RDD[U]) ⇒ RDD[(T, U)]</code> <code>mapValues</code> <code>(f : V ⇒ W)</code> : <code>RDD[(K, V)] ⇒ RDD[(K, W)]</code> <code>sort</code> <code>(c : Comparator[K])</code> : <code>RDD[(K, V)] ⇒ RDD[(K, V)]</code> <code>partitionBy</code> <code>(p : Partitioner[K])</code> : <code>RDD[(K, V)] ⇒ RDD[(K, V)]</code>
Actions	<code>count</code> <code>()</code> : <code>RDD[T] ⇒ Long</code> <code>collect</code> <code>()</code> : <code>RDD[T] ⇒ Seq[T]</code> <code>reduce</code> <code>(f : (T, T) ⇒ T)</code> : <code>RDD[T] ⇒ T</code> <code>lookup</code> <code>(k : K)</code> : <code>RDD[(K, V)] ⇒ Seq[V]</code> <code>save</code> <code>(path : String)</code> : Outputs RDD to a storage system, <i>e.g.</i>, HDFS

Table 2: Transformations and actions available on RDDs in Spark. Seq[T] denotes a sequence of elements of type T.

MPC: MapReduce计算模型的理论对应

MPC (Massively Parallel Computing)

这里是指一般意义上抽象的数据点，可以是一条图上的边，可以是一个 $\mathbb{R}^d$ 的点等等
总体要求是：假设一个点占据一个存储单位

- 输入数据： n 个“点”，任意分布在若干机器上

$0 < \alpha < 1$ 是一个常数

- 每台机器的存储至多是 $s = o(n)$ ，典型设定是 $s = n^\alpha$

- 定义机器个数为 $m = \Omega(n/s)$

m 不是独立参数，而是取决于 n 和 s

- 在一轮中： 每台机器可以发信息到任何机器、从任何机器接受信息，但每台机器的总收发量必须是 $O(s)$ 的

- 一般我们忽略 $\text{poly } \log n$ 的 factor

例如上述的机器数 m 允许 $(n/s \cdot \text{poly } \log n)$ ，
每轮每台机器总通信可以是 $O(s \cdot \text{poly } \log n)$

关于分布式算法的输出

- 输出一般也是分布式的，因为输出很大且无法存储在一台机器上
- 具体设定取决于问题，例如
 - 排序: 每个元素存储自己在排序后的rank
 - 图的匹配: 每个点存储自己匹配到的点的ID
 - 对于图上的MST: 每条输入边存一个Yes/No表示自己是否被选进MST

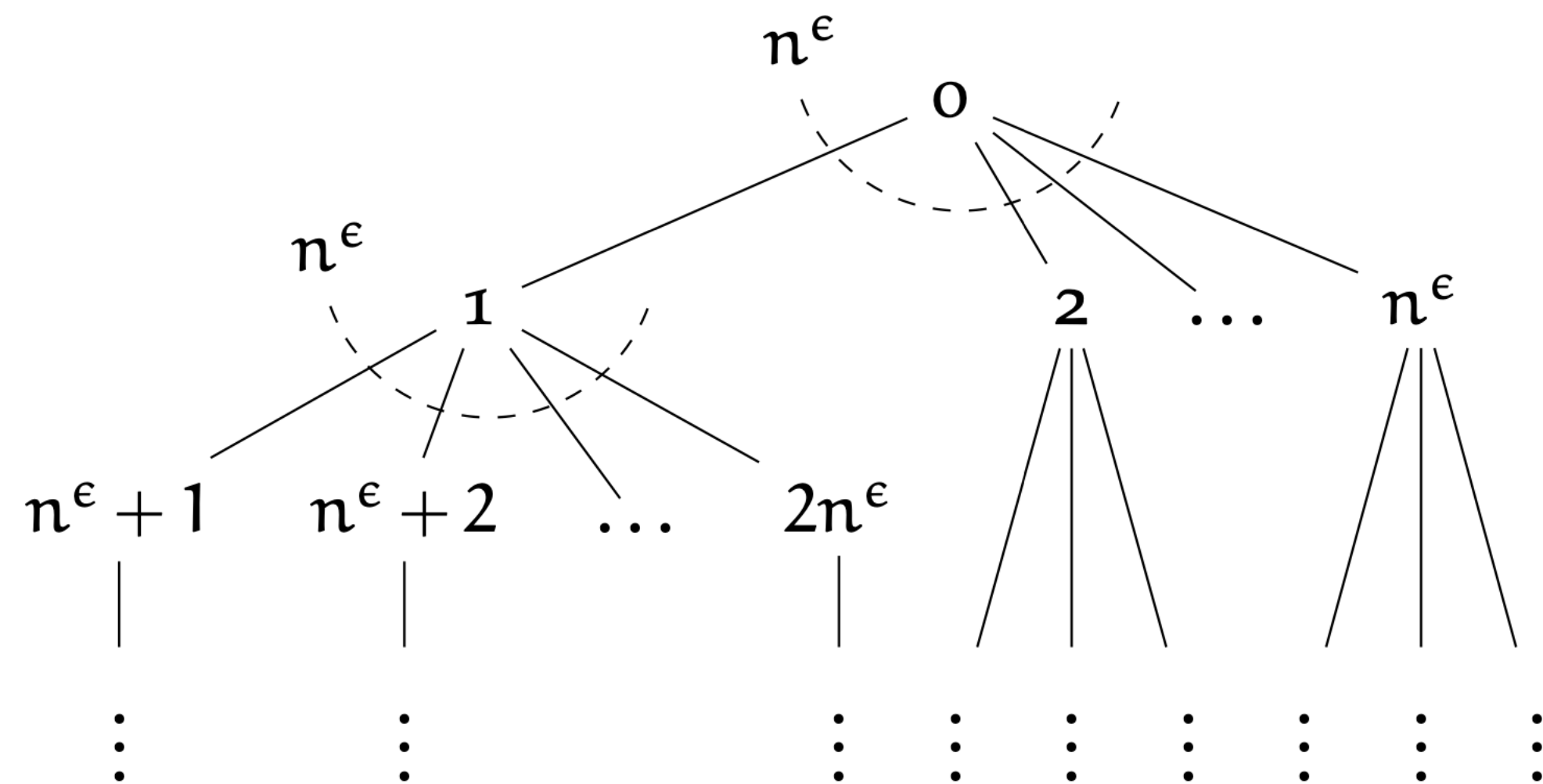
MPC算法

- 一段相同的算法/程序需要同时运行在所有机器上
- 目标：一般追求 $\text{poly } \log n$ 轮，最好是 $O(1)$ 轮
- 理论上，主要追求轮数，本地计算可以先忽略不计

但一般设计的算法的本地计算也是高效的

MPC基本算法

Broadcast广播



- 设机器1存储了一个长度为 t 的信息A
- 任务：将信息A传送/广播给所有机器
- 直接发给大家是不行的，因为会爆内存

- 设：机器内存 $s \geq t^{1+\epsilon}$

一开始，为了不爆通信限制 $O(s)$ ，机器1将长度 t 的信息A用1轮发送给 t^ϵ 个其他机器
收到信息的机器再用1轮每个发给其他 t^ϵ 个机器，以此类推

- Claim: 可在 $O(\log_s n)$ 轮完成 (考虑一个 t^ϵ -叉树进行广播)

思考：要是 $s = O(t)$ 呢？

广播的“逆”过程：Converge-cast

- Converge-cast: 从每个机器上汇总长度为 t 的信息，到一个leader（如机器1）
- 例如：
 - 每个机器有一个 t -维int向量，想要所有机器的 t -维向量的求和
- Claim: 可达到与broadcast相同的轮数

将converge-cast看作是“反向”的广播

简单应用：单点查询

- 例如，考虑 n 个分布存储在机器上的 $\mathbb{R}^d$ 上的点
- 给一个query point $p \in \mathbb{R}^d$ ，要求返回 p 在 n 个点中的 ℓ_2 最近邻
- $O(\log_s n)$ 轮可以做到：

比离线算法要容易：离线设定并不能有效支持这种暴力算法

 - 把 p broadcast给所有机器，所有机器求本地点到 p 最近邻
 - converge-cast找这些最近邻中的最近邻，汇总到机器1

事实上不止可以回答一个query point，可以回答一个batch，只要batch size $\leq s^{1-\epsilon}$

重要工具：排序

- 分布式输入 $\{a_i\}_{i=1}^n$
- 要求将 a_i 升序排列使得位次相同的元素占据连续编号的一段机器
- 结论：可以在 $O(\text{poly } \log_s n)$ 轮完成

在 $s = n^\alpha$ 的典型设定下，这个轮数就是 $O(1)$ 的

MPC实现思路： 基于Multi-pivot快速排序

- 基于multi-pivot排序方法

例如：如果输入数组是(2,3,7,5,4,10,8)且pivot是下标{2,4}，那么就需要根据 $a_2 = 3, a_4 = 5$ 来划分输入，得到值 $< 3, [3, 5], > 5$ 三个部分，即(2), (3,4,5), (7,8,10)三个新序列；之后要在三个新序列递归排序

- 如何在MPC实现？

- 找1台机器选好pivot，告知所有机器，每个机器根据pivot划分数据
- 为实现递归：将每个pivot区间汇总起来发送到连续编号的机器段上

然后在每段机器上进行递归排序

MPC实现

这里选任何 s^t ($t < 1$)都可以，是为了让broadcast在 $O(\log_s n)$ 轮可做

- 在1号机器选 $\sqrt{s}$ 个 $\{1, \dots, n\}$ 上的随机pivot，然后broadcast给所有机器
- 每台机器统计每个pivot区间有多少元素，并用converge-cast给1号机

这里pivot区间指的是例子中的三个新序列

注意到每个区间可以很大、横跨多台机器，因此需要converge-cast

- 1号机器再把得到的每个pivot区间的元素个数broadcast给所有机器
- 此时每台机器同时知道：所有pivot，和每个pivot区间长度

是一个长度是 $\sqrt{s} + 1$ 的数组/向量

MPC实现

- 回忆：此时每台机器同时知道：所有pivot，和每个pivot区间长度
- 我们想让每个pivot区间都对应一段连续编号的机器
- 机器的编号可以用区间长度来计算
- 因此每台机器：

这样所有机器都知道每个pivot区间的数据应该放到哪些个连续机器上
- 根据pivot划分本机的数据，将每个区间的数据发送到对应编号段的机器

小细节：比如一个编号段有100台机器，应该如何决定发给哪台？
难在每台机器都有一部分数据要发到这100台上，而机器间很难协调负载均衡
因此应该用随机负载均衡技术：发给这100台的随机的一台

排序轮数分析

- pivot个数是 $\sqrt{s}$ ，因此递归产生的子问题规模平均为 $n/\sqrt{s}$
 - 总递归层数 $O(\log_s n)$
- 每轮递归，所有连续的机器段内进行若干次broadcast和converge-cast
 - 这部分是 $O(\log_s n)$
- 总体上： $O(\log_s^2 n)$

Mapper和Reducer都如何在MPC实现?

本机对每个元素直接处理

- Mapper: 以单独的一对(K, V)为输入, 输出一系列(K, V)对
- Reducer: 以 K^* 的相同若干(K, V)为输入, 返回另一组同样以 K^* 为key的(K, V)

此处可以使用排序来将同一个K下的V整理到一起

MPC是一种“更灵活”的“MapReduce”模型

直接应用： Linear Regression的MPC算法

- linear regression: 输入是 $X \in \mathbb{R}^{n \times d}$
 - 以分布式稀疏方式给出: (i, j, X_{ij})
- 如果假定 d 很小而 n 很大, 具体来说 $\text{poly}(d) \ll s \ll n$
- 我们学过的方法: 先降维, 把 n 降成大概 $\text{poly}(d)$
 - 然后在 $\text{poly}(d) \times d$ 矩阵 $\hat{X}$ 上用公式 $w = (\hat{X}^\top \hat{X})^{-1} \hat{X}^\top \hat{y}$

回忆：降维矩阵S

设随机 $S \in \mathbb{R}^{m \times n}$ 每列只有一个均匀随机位置非0，且该元素取均匀随机 $\{-1, 1\}$

$$S = \begin{pmatrix} 0 & 0 & \cdots & 1 & 0 & -1 \\ \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 1 & \vdots & \cdots & \vdots & \vdots & \vdots \\ \vdots & 1 & \cdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & -1 & 0 \end{pmatrix}$$

[Clarkson-Woodruff, STOC 13]

定理：用 $A' = S$ 替代JL矩阵A， $m = O(\epsilon^{-2} d^2 \cdot \text{poly} \log(\epsilon^{-1} d))$ 有类似保证

$$\forall v \in \mathcal{U}, \quad \Pr[\|A'v\|^2 \in (1 \pm \epsilon) \cdot \|v\|^2] \geq 0.9$$

回忆 $\mathcal{U}$ 是 $\mathbb{R}^n$ 的 d 维子空间

如何在MPC计算 SX ?

- 对 $i \in [n]$, 设 S 的第 i 列选取的随机行为 $\pi(i)$
- 性质: 设 $M = SX$, 则 X_{ij} 只会影响 $M_{\pi(i),j}$
 π 需要 $\Omega(n)$ 空间存储, 无法预先在一台机器生成, 只能在每个 i 的那一组分布式生成
- MPC算法: 将 (i, j, X_{ij}) 按照 i 为关键字排序, 计算 $\pi(i)$ 并broadcast给 $(i, \cdot, \cdot)$
轮数?
- 再按 $\pi(i)$ 为第一关键字、 j 为第二关键字排序 (i, j, X_{ij})
此时排序后的一段可汇总出 $M_{\pi(i),j}$ 的值
- 最后再将 M 直接发送到机器1, 进行本地求解 $w = (\hat{X}^\top \hat{X})^{-1} \hat{X}^\top \hat{y}$

矩阵大小 $\text{poly}(d)$, 根据假设可以放在一台机器上

MPC上的算法研究

MPC算法研究概述

- MPC算法是新兴算法领域，与大数据处理的关系密切
 - 也受到了以Google Research为代表的工业界实验室的大力推动
- 一个研究主线是为各种不同输入数据的设定和问题设计MPC算法：
 - 图数据
 - $\mathbb{R}^d$ 数据
 - 字符串数据等

根本挑战和细分设定

- 根本挑战：sublinear的空间
- 又称为fully scalable：即算法需要可以处理任何 $s = n^\epsilon$ ，且最好是 $O_\epsilon(1)$ 轮
- 前述的基本算法都是fully scalable的
- 对于一般问题：fully scalable经常很困难，有时根据问题参数考虑细分设定

即允许使用任意弱的机器

图数据的MPC算法：细分Regime

- 图数据：设有 n 个顶点，输入是图的边集

数据至多可以是 $O(n^2)$ 大小

- 注意到输入规模其实是边数 m ，而不是 n
- 根据机器空间 s 与 n 的关系分为三个regime：

连点集和问题的解都存不下

- super-linear: $s = n^{1+\epsilon}$, near-linear: $s = \tilde{O}(n)$, sub-linear: $s = n^{1-\epsilon}$

能存下点集和典型问题，例如MST/matching的解
还能有 n^ϵ 空余做别的

能存下点集和很多典型问题的解
但是没有很多额外空间做其他事情

最小生成树MST的MPC算法

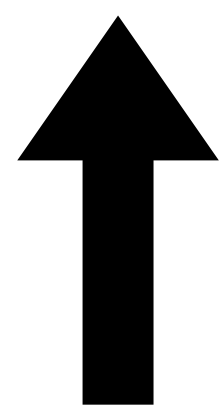
Super-linear Regime下的MST

Filtering技术 [Karloff, Suri, Vassilvitskii, SODA 2010]

边数太少没意义：基本上可以发到一台机器上运行经典算法解决掉

- 空间 $s = \tilde{O}(n^{1+\epsilon})$ ，且设 $|E| = m = n^{1+c}$
- 结论：存在MPC算法能在 $O(c/\epsilon)$ 轮计算最小生成树的精确解

细节：一轮converge-cast之后，会收集到 n^ϵ 个MST边集的并。此时，需要把这个并集的MST求出来，再继续converge cast



Algorithm: MST(V,E)

η 就是我们的 s

```
1: if  $|E| < \eta$  then
2:   Compute  $T^* = MST(E)$ 
3:   return  $T^*$ 
4: end if
5:  $\ell \leftarrow \Theta(|E|/\eta)$ 
6: Partition  $E$  into  $E_1, E_2, \dots, E_\ell$  where  $|E_i| < \eta$  using a
   universal hash function  $h : E \rightarrow \{1, 2, \dots, \ell\}$ .
7: In parallel: Compute  $T_i$ , the minimum spanning tree on
    $G(V, E_i)$ .
8: return  $MST(V, \cup_i T_i)$ 
```

如果当前边数少于 s 了，就可以发到一台机器解决

否则，把边集均匀分配到每台机器上，在每台机器上收到的边集 E_i 用传统算法计算MST

最后需要一个converge-cast来汇总每台机器上的MST

Near-linear Regime下的MST

Linear Sketching技术

- 这里介绍一个一般方法：数据流算法经常可以转成MPC算法！
- 回忆：对于图数据流，我们得到了一个 $\tilde{O}(n/\epsilon)$ 空间的 $(1 + \epsilon)$ 近似MST的算法
- 具体来说：我们对每个顶点 v 维护了总大小 $\tilde{O}(1)$ 的若干 ℓ_0 -采样结构
- 重点：这些 ℓ_0 -采样结构都是linear-sketch

回忆linear sketch重要性质：可合并

具体实现

回忆：排序操作可以在 $O(\log_s n)$ 轮完成

- 将边按照较小顶点号排序
 - 这样与每个顶点邻接的边都放在了某同一台机器上
- 对于每个顶点 v ，将邻接边插入streaming算法维护的 $\tilde{O}(1)$ 大小的一众 ℓ_0 结构里
- 把每个顶点的数据结构直接发到1号机器
 - 注意这里不需要converge-cast来汇总，并且不会爆空间（为什么？）
- 此时1号机器就有了整个图的streaming数据结构，可直接运行查询得到MST

类似的：欧氏空间近似直径

- 回忆：数据流sparse recovery算法可在 $O(\epsilon^{-d} \text{poly } \log n)$ 空间 $(1 + \epsilon)$ 近似直径
- MPC算法：
 - 机器将本机点集喂给该数据流算法，得到一个 $O(\epsilon^{-d} \text{poly } \log n)$ 大小的结构
 - 使用converge-cast把所有的结构“求和”，汇总到机器1
 - 机器1继续运行求直径的算法，得出直径

因为是linear sketch，所以可以相加

根本挑战： Sub-linear Regime

MST的结果

- 一般图仍只有最坏情况 $o(\log n)$ -轮的 $(1 + \epsilon)$ -近似
 - 该设定也有 $\Omega(\log n)$ 轮的conditional lower bound
- 如果input满足三角形不等式 (metric space)
 - 一般metric有 $O(\log(\epsilon^{-1}) + \log \log n)$ -轮的 $(1 + \epsilon)$ -近似
[Azarmehr, Behnezhad, Jayaram, Łącki, Mirrokni, Zhong, SODA 25]
 - 低维欧氏有 $O(1)$ -轮的 $(1 + \epsilon)$ -近似, 但local space要求 $2^{\Omega(d)}$
[Andoni, Nikolov, Onak, Yaroslavtsev, STOC 14]

Sublinear/Fully Scalable Regime

Sub-linear Regime的最基本问题： MIS

- MIS = **Maximal** Independent Set
- 输入：无向图 $G(V, E)$
- IS是一个 $S \subseteq V$ such that $\forall x \neq y \in S$ 有 $\{x, y\} \notin E$
- MIS就是极大的IS，即无法加入任何点 $u \in V$ 使该集合继续是IS
- 特点：计算一组解天然需要很多（轮）通信，但验证一组解是简单的

经典算法：Luby

[Luby, SICOMP 1986]

- 本来定义在Local Model上
- Local Model:
 - 输入网络拓扑图 $G(V, E)$, 每个vertex有一台机器, 该机器知道自己的邻居信息
 - 每台机器每轮可以向所有直接邻居发送任意长度的信息
- **Claim:** 在Local Model下, $O(\log n)$ -轮可以计算一个拓扑图的MIS
 - “计算”的意思: 每台机器赋予Yes或者No, 代表是否选入MIS

Luby算法过程

1. 初始化标记所有点un-visited, MIS从属为No
2. for $i = 1, \dots, O(\log n)$
3. 每台un-visited机器 j 生成一个 $[0,1]$ 上的均匀随机数 h_j
该规则标记Yes的点构成IS
4. 每台un-visited机器 $j \in P$ 若 $h_j = \min\{h_k : k \in P \cap N(j)\}$ 则赋 j 为Yes
5. 每台un-visited机器 j 若 $\{j\} \cup N(j)$ 有赋Yes的, 则标记 j 为visited
可以理解成MIS的点及其邻居都在本轮for循环“删除”

主要Claim: a) 算法每轮找到的一定是MIS; b) 每轮期望会删掉一半的边

Luby的MPC实现

- Claim: Luby的每次for循环可以用 $O(1)$ 个MPC轮实现
 - 随机数生成可在1轮完成
 - 其余步骤:
 - 每条无向边 $\{u, v\}$ 复制成有向边 (u, v) 和 (v, u)
 - 将有向边按照第一顶点为关键字排序
 - 连续段对应一个顶点的邻居, 可直接进行统计 (converge-cast etc)

MIS的Fully Scalable MPC算法： 目前进展

- Luby的算法依然是最坏情况下已知最好的
- 有一些conditional lower bound结果一定程度上说 $\log n$ 不可以显著改进
- 对特例可能做得更好，例如bounded degree图 [\[Ghaffari, SODA 16\]](#)
- 更多近期进展参考Mohsen Ghaffari的工作

聚类问题的Fully Scalable MPC算法

- 聚类问题: k -median, k -means, k -center
- 若允许本地空间 $s \geq \Omega(k)$, 则较为容易、存在 $O(1)$ 轮 $(1 + \epsilon)$ -近似
并不 fully scalable
- Fully scalable 并且 $O(1)$ 轮的算法:
 - k -median: $O(\log^{1.5})$ 近似 [Cohen-Addad, Lattanzi, Norouzi-Fard, Sohler, Svensson, NeurIPS 21]
[Ahanchi, Andoni, Hajiaghayi, Knittel, Zhong, SPAA 23]
 - k -center: 低维 2-近似, 高维 $O(\log n / \log \log n)$ -近似
[Czumaj, Gao, Ghaffari, J, ICALP 25]
 - k -means: 即使 2 维也没有结果 高维 $O((\log n / \log \log n)^2)$ -近似
[J, Jin, Lou, Wang, arXiv]

研究方向：MPC上的几何问题

- 很多非常基本的计算问题都没有得到研究，即使对于低维/2维
 - 举例：最近点对，全局最小权匹配，凸包.....
- 高维：
 - 什么都不知道，包括直径、最小生成树这些问题能做到什么程度都是open的
- 我正在考虑的问题：高维的直径、MST，聚类